

Recursion

- The process in which a function calls itself directly or indirectly is called **recursion** and the corresponding function is called a **recursive function**.

```
10 + sum(9)
10 + ( 9 + sum(8) )
10 + ( 9 + ( 8 + sum(7) ) )
...
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + sum(0)
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0
```

```
int sum(int k) {
    if (k > 0) {
        return k + sum(k - 1);
    } else {
        return 0;
    }
}
```

```
int main() {
    int result = sum(10);
    cout << result;
    return 0;
}
```

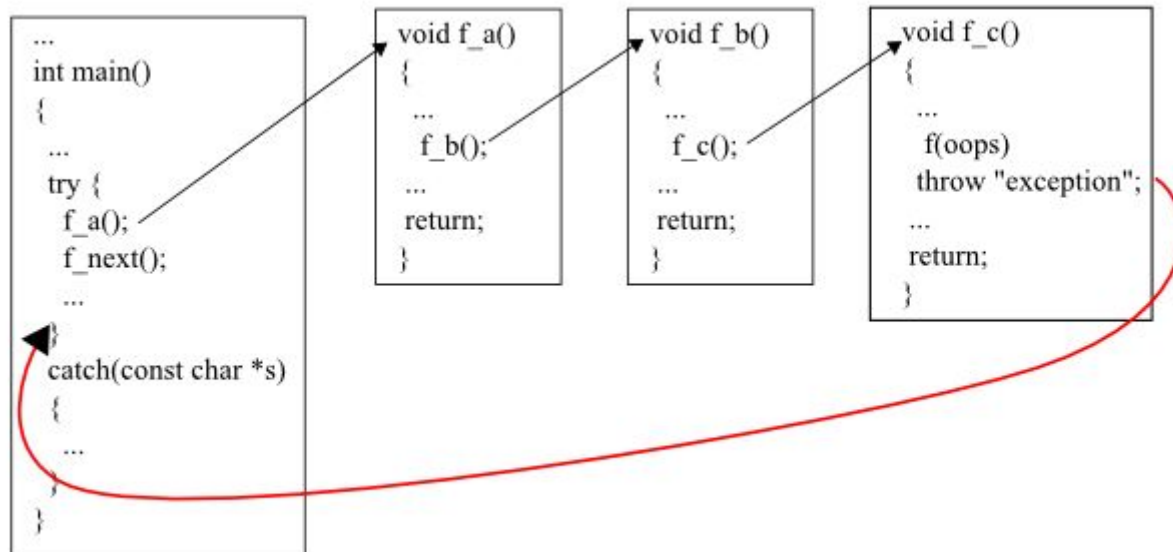
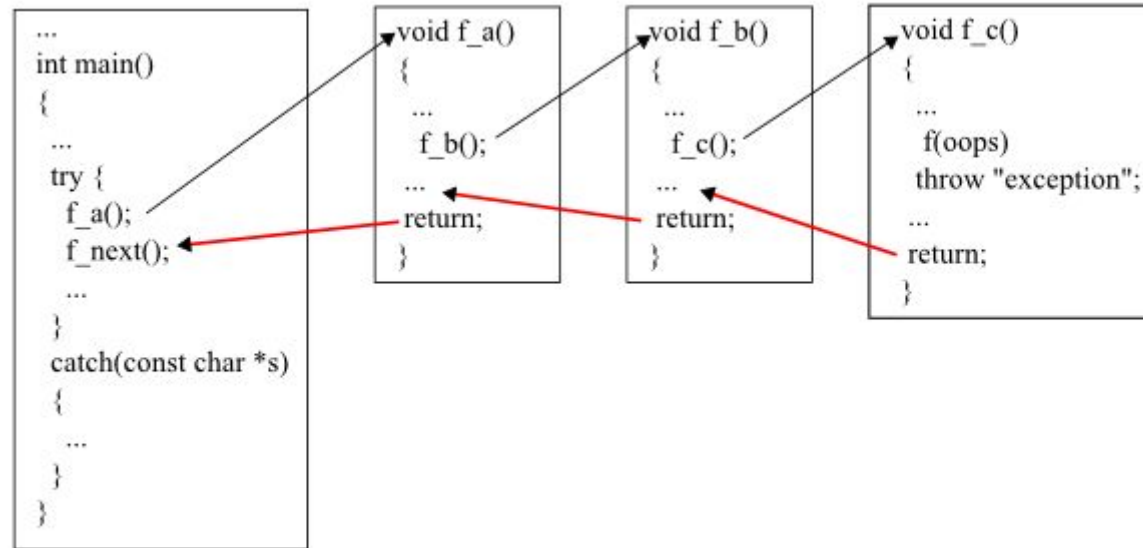
Initial Call

```
sum(10)
sum(9)
sum(8)
sum(7)
sum(6)
sum(5)
sum(4)
sum(3)
sum(2)
sum(1)
sum(0)
```

Unwinding

```
sum(0) returns 0
sum(1) returns 1
sum(2) returns 3
sum(3) returns 6
sum(4) returns 10
sum(5) returns 15
sum(6) returns 21
sum(7) returns 28
sum(8) returns 36
sum(9) returns 45
sum(10) returns 55
```

- Initial Call: The first invocation of the function (e.g., `sum(10)`) that triggers the recursion.
- Unwinding: The process of resolving each recursive call from the base case back to the original call, combining results and calculating the final value.



Example#01 – Sum of list of numbers

```
4 ▾ int sum_list_loop(const int numbers[], int size) {  
5     int total = 0;  
6 ▾     for (int i = 0; i < size; ++i) {  
7         total += numbers[i];  
8     }  
9     return total;  
10 }
```

```
4 ▾ int sum_list_recursive(const int numbers[], int size) {  
5 ▾     if (size == 0) {  
6         return 0;  
7 ▾     } else {  
8         return numbers[size - 1] + sum_list_recursive(numbers, size - 1);  
9     }  
10 }  
11
```

Recursion	Iteration
Terminates when the base case becomes true.	Terminates when the condition becomes false.
Used with functions.	Used with loops.
Every recursive call needs extra space in the stack memory.	Every iteration does not require any extra space.
Smaller code size.	Larger code size.

Example#02 – factorial

```
4 int factorial(int n)
5 {
6     if (n == 0 || n == 1) {
7         return 1;
8     } else {
9         return n * factorial(n - 1);
10    }
11 }
```

For user input : 5

Factorial Recursion Function

$n * f(n-1)$

Final Result

$5 * f(4) = 5 * 24 = 120$

$4 * f(3) = 4 * 6 = 24$

$3 * f(2) = 3 * 2 = 6$

$2 * f(1) = 2 * 1 = 2$

Advantages / Disadvantages

- A lot of algorithms are naturally defined as recursive steps. Using recursion will make these program more clear to understand and elegant to review.
- Recursion is memory hungry and a bit slow, you need to have a good reason to use recursion.
- In computer science-recursion is generally supported for readability, understandability and clean code.

Classification of Recursion

- Tail vs. Non-Tail
- Direct vs. Indirect vs. Nested
- Simple vs. Excessive
- Nested Recursion

Tail Recursion

- Tail Recursion is characterized by the use of only one recursive call at the very end of a function implementation.
- This means when a recursive call is invoked there is no work left for the previous call.
- Moreover, there is no pending recursive calls, either direct or indirect.

```
void tail(int i) {  
    if (i > 0) {  
        cout << i << ' ';  
        tail(i-1);  
    }  
}
```

```
void nonTail(int i) {  
    if (i > 0) {  
        nonTail(i-1);  
        cout << i << ' ';  
        nonTail(i-1);  
    }  
}
```

Non-Tail Recursion

- Non-Tail recursion is characterized by the fact that there is still some work left for the first call and a recursive self call is invoked.

```
/* 200 */ void reverse() {  
    char ch;  
/* 201 */    cin.get(ch);  
/* 202 */    if (ch != '\n') {  
/* 203 */        reverse();  
/* 204 */        cout.put(ch);  
    }  
}
```

Indirect Recursion

- Indirect Recursion is characterized by two or more distinct functions repeatedly call each other. Hence a recursive phenomena is there.

```
int Fun1(int & n)
{
    if (n <= 20) // do not invoke Fun 2 once n reaches 20
    {
        cout<< n << endl;
        n++;
        Fun2(n);
    }
    else return 0;
}

int Fun2(int & n)
{
    if (n <= 20) // do not invoke Fun1 once n reaches 20
    {
        cout<< n << endl;
        n++;
        Fun1(n);
    }
    else return 0;
}
```

Excessive Recursion

- Generally while solving a maze, the allowable movements are left, right, up, down (diagonal also allowed), 4 (8) in number.

```
with Start row, col
    FindPath(maze,row, col+1);
    FindPath(maze,row, col-1);
    FindPath(maze,row-1, col);
    FindPath(maze,row+1, col);
```

Nested Recursion

- When a recursive function call is embedded into a recursive function, it is called “Nested Recursion”
- Example of nested recursion is Ackerman function:

$A(m,n) = n + 1$	if $m == 0$
$A(m,n) = A(m-1, 1)$	if $m > 0 \ \&\& \ n == 0$
$A(m,n) = A(m-1, A(m, n-1))$	if $m > 0 \ \&\& \ n > 0$

- For example, $A(4, 2)$ is an integer of 19,729 decimal digits